

Resumo do Modulo 2 - Programacao Orientada a Objetos (POO)

1. Classe

E como criar um molde de algo.

```
class Pessoa:  
    def __init__(self, nome):  
        self.nome = nome
```

2. Objeto

E a instancia da classe. Tipo: 'usei o molde e criei uma pessoa'.

```
lucas = Pessoa("Lucas")
```

3. Metodo

E uma funcao dentro da classe.

```
def falar(self):  
    print("Ola!")
```

4. Heranca

Permite criar uma classe que herda de outra.

```
class Guerreiro(Personagem):  
    def atacar(self):  
        return "Espadada!"
```

5. Polimorfismo

Mesma funcao, comportamento diferente.

```
guerreiro.atacar() # Espadada!  
mago.atacar()      # Bola de fogo!
```

6. Encapsulamento

Controla o acesso aos atributos da classe usando underline (_).

7. Classe Abstrata

Serve como modelo. Nao pode ser usada sozinha.

```
class Personagem(ABC):  
    @abstractmethod  
    def atacar(self):  
        pass
```

8. Heranca Multipla

Resumo do Modulo 2 - Programacao Orientada a Objetos (POO)

Uma classe herda de mais de uma ao mesmo tempo.

```
class Morcego(Mamifero, Ave):
```

9. Decoradores

Funcoes que embrulham outras funcoes pra adicionar comportamento.

```
@verifica_senha  
def acessar():  
    ...
```

Por que parece mais dificil?

Porque agora o foco nao e mais o que o codigo faz, mas como organizar, reutilizar e manter ele vivo em projetos maiores.

Como pensar em POO:

1. Quais 'coisas' existem no meu sistema? -> vira classe (Personagem, Conta, Produto)
2. Quais acoes elas fazem? -> vira metodo (atacar(), sacar(), exhibir())
3. Quais atributos cada uma tem? -> vira variavel (nome, vida, saldo)
4. Tem alguma classe que pode herdar de outra? -> heranca
5. As acoes sao parecidas, mas mudam o jeito? -> polimorfismo